

Fase di Elaborazione - Iterazione 1

Gestore di Ordinazioni per Fast Food

Scala Salvatore Gaetano

Ingegneria del Software A-Z

Prof. Orazio Tomarchio

Anno Accademico 2025/2026

Indice

1	Elaborazione - Iterazione 1	2
1.1	Introduzione	2
1.2	Analisi Orientata agli Oggetti	2
1.2.1	Modello di Dominio	2
1.2.2	Diagramma di Sequenza di Sistema (SSD)	4
1.2.3	Contratti delle operazioni	4
1.3	Progettazione Orientata agli Oggetti	6
1.3.1	Diagrammi di Interazione (Sequenza)	6
1.3.2	Diagramma delle Classi di Progetto (DCD)	7
1.4	Documentazione dei Casi di Test	8

Capitolo 1

Elaborazione - Iterazione 1

1.1 Introduzione

Conclusa la fase di ideazione, si passa alla fase di elaborazione. Lo scopo di questa prima iterazione è implementare in maniera iterativa il nucleo dell'architettura del software, definendo gli oggetti principali del dominio e le loro responsabilità. Durante questa prima iterazione i requisiti scelti su cui concentrarsi sono:

- Implementare lo scenario principale di successo del caso d'uso **UC1: Gestisci Ordinazione**.
- Tralasciare in questa iterazione, per semplicità, l'associazione delle promozioni all'ordine e la gestione in tempo reale della coda in cucina (che verranno affrontate nell'Iterazione 2).
- I dati, per questa fase, verranno gestiti in memoria principale.

1.2 Analisi Orientata agli Oggetti

L'analisi orientata agli oggetti si basa sulla creazione di una descrizione del dominio del problema. Vengono utilizzati tre strumenti principali: il Modello di Dominio, i Diagrammi di Sequenza di Sistema (SSD) e i Contratti delle operazioni.

1.2.1 Modello di Dominio

Relativamente al caso d'uso scelto (UC1), dopo un'attenta valutazione dello scenario principale di successo, è stato possibile identificare le seguenti classi concettuali:

- **Ordine:** rappresenta l'insieme dei prodotti richiesti dal cliente e il totale da pagare.
- **VoceOrdine:** contiene i dettagli relativi all'aggiunta di un particolare prodotto, la sua quantità e le personalizzazioni.
- **Prodotto:** rappresenta l'articolo base presente nel menu (es. Hamburger).
- **Personalizzazione:** rappresenta l'aggiunta o rimozione di un ingrediente rispetto alla ricetta base.

- **Catalogo:** contiene tutte le descrizioni dei prodotti acquistabili.
- **Cliente:** attore primario, che interagisce con il sistema.
- **Chiosco (Sistema):** rappresenta il sistema "Gestore Ordinanze Fast Food".
- **Pagamento:** rappresenta la transazione economica simulata.

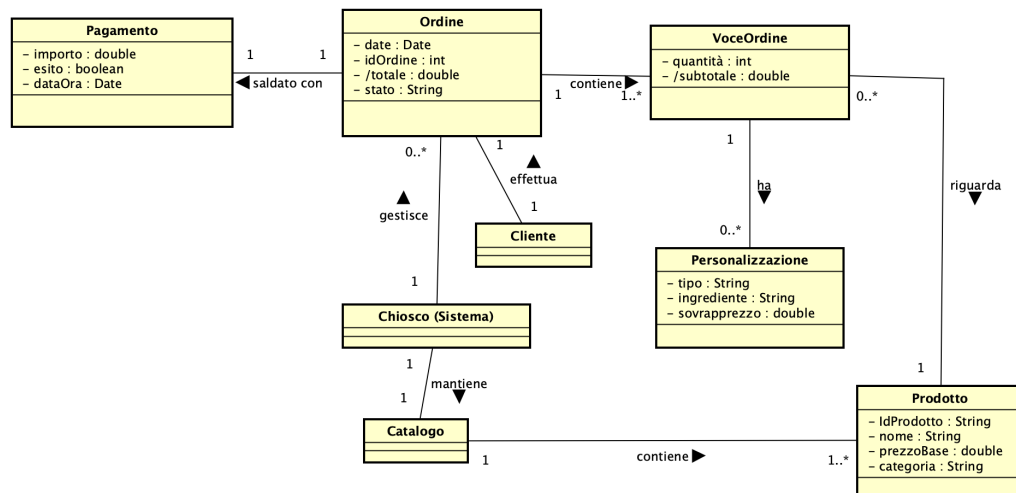


Figura 1.1: Modello di Dominio - Iterazione 1

1.2.2 Diagramma di Sequenza di Sistema (SSD)

Il passo successivo è la creazione del Diagramma di Sequenza di Sistema (SSD) al fine di illustrare il corso degli eventi di input e di output per lo scenario principale di successo di UC1.

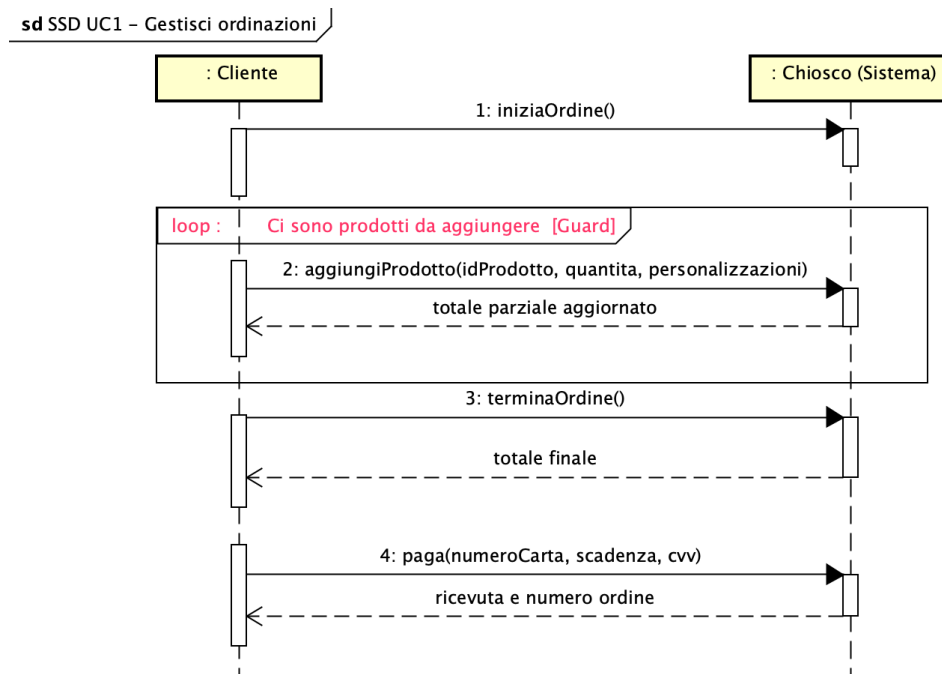


Figura 1.2: SSD per UC1: Gestisci Ordinazione

Gli eventi di sistema individuati nell'SSD per interagire con il *:Sistema* sono:

- `iniziaOrdine()`
- `aggiungiProdotto(idProdotto, quantita, personalizzazioni)`
- `terminaOrdine()`
- `paga(numeroCarta, scadenza, cvv)`

1.2.3 Contratti delle operazioni

Vengono ora descritte attraverso i Contratti le principali operazioni di sistema che si occupano di gestire gli eventi individuati nell'SSD.

Contratto CO1: iniziaOrdine

Operazione:	<code>iniziaOrdine()</code>
Riferimenti:	Caso d'uso: Gestisci Ordinazione

Pre-condizioni:	Il chiosco è in attesa di un nuovo cliente.
Post-Condizioni:	- È stata creata una nuova istanza <i>o</i> di Ordine. - L'istanza <i>o</i> è stata associata al Sistema Chiosco come ordine corrente.

Contratto CO2: aggiungiProdotto

Operazione:	aggiungiProdotto(idProdotto: String, quantita: int, personalizzazioni: List<String[]>)
Riferimenti:	Caso d'uso: Gestisci Ordinazione
Pre-condizioni:	È in corso un ordine corrente.
Post-Condizioni:	- È stata creata una nuova istanza <i>vo</i> di VoceOrdine. - Gli attributi di <i>vo</i> sono stati aggiornati con la quantità. - <i>vo</i> è stata associata all'Ordine corrente tramite associazione "contiene". - <i>vo</i> è stata associata al Prodotto corrispondente a <i>idProdotto</i>. - Sono state create eventuali istanze di <i>Personalizzazione</i> associate a <i>vo</i>.

Contratto CO3: terminaOrdine

Operazione:	terminaOrdine()
Riferimenti:	Caso d'uso: Gestisci Ordinazione
Pre-condizioni:	È in corso un ordine con almeno una VoceOrdine.
Post-Condizioni:	- L'attributo totale dell'Ordine corrente è stato calcolato e aggiornato.

Contratto CO4: paga

Operazione:	paga(numeroCarta: String, scadenza: String, cvv: String)
Riferimenti:	Caso d'uso: Gestisci Ordinazione
Pre-condizioni:	L'ordine è terminato e il totale è stato calcolato.
Post-Condizioni:	- È stata creata un'istanza <i>p</i> di Pagamento. - <i>p</i> è stata associata all'Ordine corrente. - L'attributo stato dell'Ordine è stato aggiornato a "Ricevuto". - L'Ordine è stato salvato nello storico del Sistema.

1.3 Progettazione Orientata agli Oggetti

La progettazione orientata agli oggetti definisce le responsabilità degli oggetti software e come questi collaborano. In questa fase applichiamo i pattern **GRASP** (come Controller, Creator, Information Expert) per allocare correttamente i metodi alle classi.

1.3.1 Diagrammi di Interazione (Sequenza)

(Nota: I diagrammi di sequenza mostreranno come il Chiosco, agendo da Facade Controller, delega la creazione della VoceOrdine all'oggetto Ordine in base al pattern Creator, e come l'Ordine calcola il totale richiedendo i sub-totali alle singole VociOrdine per il pattern Information Expert).

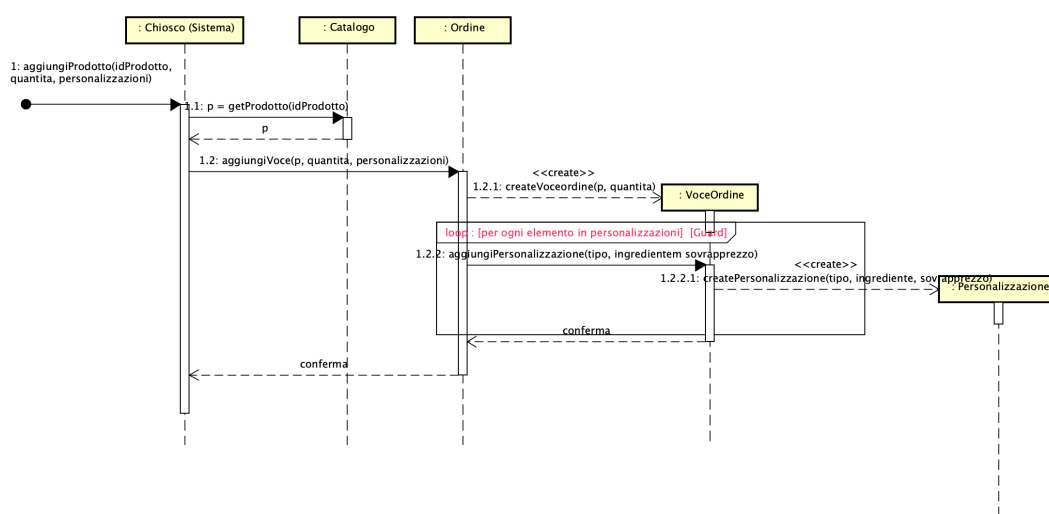


Figura 1.3: Diagramma di Sequenza: aggiungiProdotto

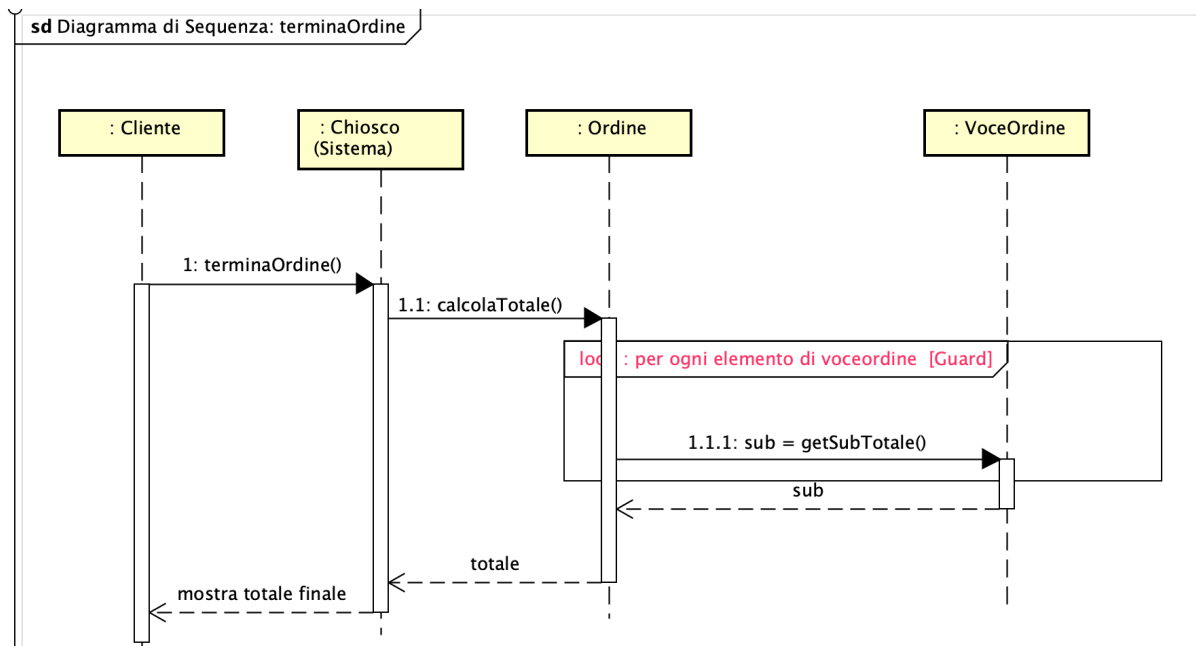


Figura 1.4: Diagramma di Sequenza: terminaOrdine

1.3.2 Diagramma delle Classi di Progetto (DCD)

Il Diagramma delle Classi di Progetto (DCD) evolve dal Modello di Dominio, integrando i metodi ricavati dai diagrammi di sequenza, la visibilità degli attributi e le relazioni di navigabilità.

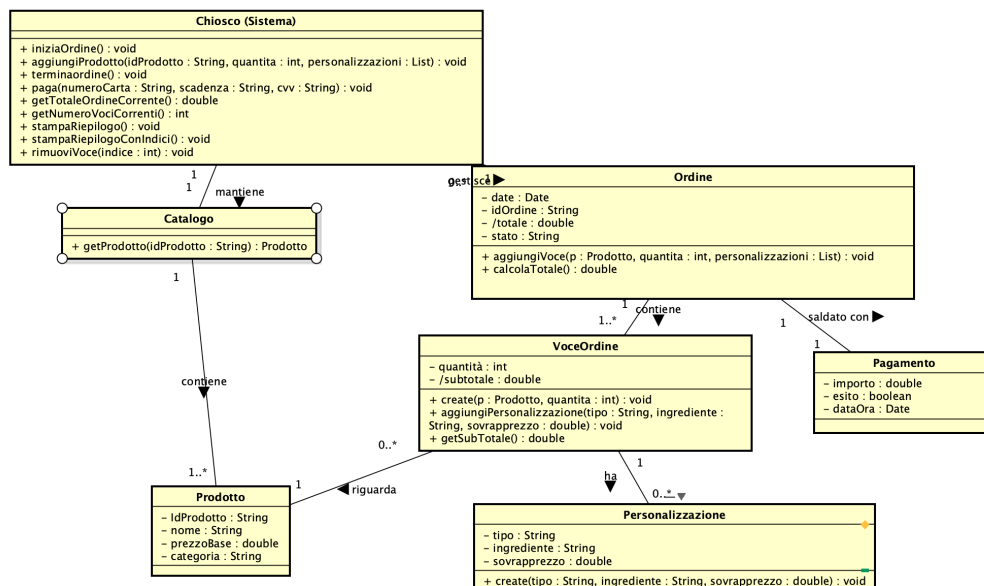


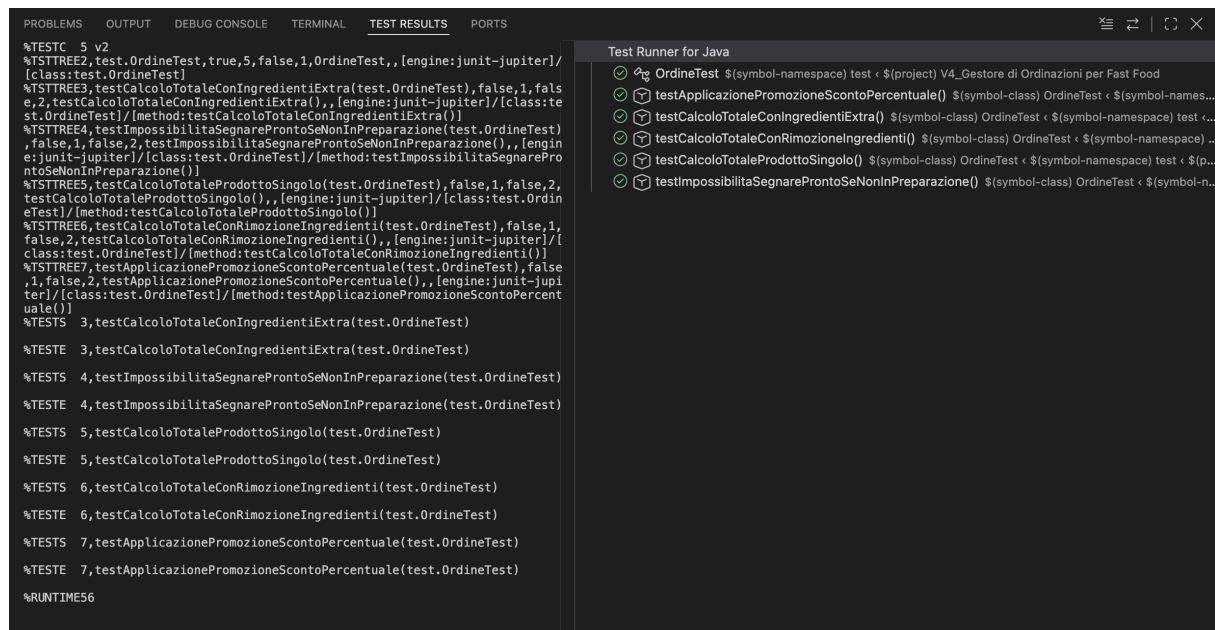
Figura 1.5: Design Class Diagram (DCD) - Iterazione 1

1.4 Documentazione dei Casi di Test

Come richiesto dalle specifiche di progetto, sono stati implementati casi di test utilizzando il framework **jUnit**. La strategia di identificazione dei test si è basata sul testing funzionale (Black Box), individuando le classi di equivalenza per le funzioni critiche del dominio. In particolare, è stata testata accuratamente la classe `Ordine` e il metodo `calcolaTotale()`, verificando:

- Il calcolo corretto del totale con un solo prodotto senza personalizzazioni.
- Il calcolo corretto del totale con l'aggiunta di ingredienti extra (sovrapprezzo applicato).
- Il calcolo corretto del totale con la rimozione di ingredienti standard (nessuna deduzione di prezzo applicata, in base alla Regola di Business R2).

I casi di test documentati sono stati automatizzati e l'esecuzione della test suite ha dato esito pienamente positivo (con una percentuale di superamento del 100%). Questo conferma la solidità della logica di business e il corretto funzionamento delle classi di dominio, come si evince dallo screenshot dell'ambiente di sviluppo riportato di seguito:



```
%TESTC 5 v2
%TSTTREE2, testOrdineTest, true, 5, false, 1, OrdineTest, [[engine:junit-jupiter]/[class:test.OrdineTest]]
%TSTTREE3, testCalcoloTotaleConIngredientiExtra(test.OrdineTest), false, 1, false, 2, testCalcoloTotaleConIngredientiExtra(), [[engine:junit-jupiter]/[class:test.OrdineTest]/[method:testCalcoloTotaleConIngredientiExtra()]]
%TSTTREE4, testImpossibilitaSegnareProntoSeNonInPreparazione(test.OrdineTest), false, 1, false, 2, testImpossibilitaSegnareProntoSeNonInPreparazione(), [[engine:junit-jupiter]/[class:test.OrdineTest]/[method:testImpossibilitaSegnareProntoSeNonInPreparazione()]]
%TSTTREE5, testCalcoloTotaleProdottoSingolo(test.OrdineTest), false, 1, false, 2, testCalcoloTotaleProdottoSingolo(), [[engine:junit-jupiter]/[class:test.OrdineTest]/[method:testCalcoloTotaleProdottoSingolo()]]
%TSTTREE6, testCalcoloTotaleConRimozioneIngredienti(test.OrdineTest), false, 1, false, 2, testCalcoloTotaleConRimozioneIngredienti(), [[engine:junit-jupiter]/[class:test.OrdineTest]/[method:testCalcoloTotaleConRimozioneIngredienti()]]
%TSTTREE7, testApplicazionePromozioneScontoPercentuale(test.OrdineTest), false, 1, false, 2, testApplicazionePromozioneScontoPercentuale(), [[engine:junit-jupiter]/[class:test.OrdineTest]/[method:testApplicazionePromozioneScontoPercentuale()]]
%TESTS 3, testCalcoloTotaleConIngredientiExtra(test.OrdineTest)
%TESTE 3, testCalcoloTotaleConIngredientiExtra(test.OrdineTest)
%TESTS 4, testImpossibilitaSegnareProntoSeNonInPreparazione(test.OrdineTest)
%TESTE 4, testImpossibilitaSegnareProntoSeNonInPreparazione(test.OrdineTest)
%TESTS 5, testCalcoloTotaleProdottoSingolo(test.OrdineTest)
%TESTE 5, testCalcoloTotaleProdottoSingolo(test.OrdineTest)
%TESTS 6, testCalcoloTotaleConRimozioneIngredienti(test.OrdineTest)
%TESTE 6, testCalcoloTotaleConRimozioneIngredienti(test.OrdineTest)
%TESTS 7, testApplicazionePromozioneScontoPercentuale(test.OrdineTest)
%TESTE 7, testApplicazionePromozioneScontoPercentuale(test.OrdineTest)
%RUNTIME56
```

Figura 1.6: Esito dell'esecuzione della Test Suite jUnit